Week 4 Lesson 2 — K-Nearest Neighbors

11th November 2026

1 K Nearest Neighbors | Intuitive explained | Machine Learning Basics

From the Youtube video: K Nearest Neighbors | Intuitive explained | Machine Learning Basics

[1]:
```
# knn from sklearn
from sklearn import neighbors, datasets

# import some data to play with
iris = datasets.load_iris()

iris
```

[1]:
```
{'data': array([[5.1, 3.5, 1.4, 0.2],
        [4.9, 3. , 1.4, 0.2],
        [4.7, 3.2, 1.3, 0.2],
        [4.6, 3.1, 1.5, 0.2],
        [5. , 3.6, 1.4, 0.2],
        [5.4, 3.9, 1.7, 0.4],
        [4.6, 3.4, 1.4, 0.3],
        [5. , 3.4, 1.5, 0.2],
        [4.4, 2.9, 1.4, 0.2],
        [4.9, 3.1, 1.5, 0.1],
        [5.4, 3.7, 1.5, 0.2],
        [4.8, 3.4, 1.6, 0.2],
        [4.8, 3. , 1.4, 0.1],
        [4.3, 3. , 1.1, 0.1],
        [5.8, 4. , 1.2, 0.2],
        [5.7, 4.4, 1.5, 0.4],
        [5.4, 3.9, 1.3, 0.4],
        [5.1, 3.5, 1.4, 0.3],
        [5.7, 3.8, 1.7, 0.3],
        [5.1, 3.8, 1.5, 0.3],
        [5.4, 3.4, 1.7, 0.2],
        [5.1, 3.7, 1.5, 0.4],
        [4.6, 3.6, 1. , 0.2],
        [5.1, 3.3, 1.7, 0.5],
```

```
[4.8, 3.4, 1.9, 0.2],
[5. , 3. , 1.6, 0.2],
[5. , 3.4, 1.6, 0.4],
[5.2, 3.5, 1.5, 0.2],
[5.2, 3.4, 1.4, 0.2],
[4.7, 3.2, 1.6, 0.2],
[4.8, 3.1, 1.6, 0.2],
[5.4, 3.4, 1.5, 0.4],
[5.2, 4.1, 1.5, 0.1],
[5.5, 4.2, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.2],
[5. , 3.2, 1.2, 0.2],
[5.5, 3.5, 1.3, 0.2],
[4.9, 3.6, 1.4, 0.1],
[4.4, 3. , 1.3, 0.2],
[5.1, 3.4, 1.5, 0.2],
[5. , 3.5, 1.3, 0.3],
[4.5, 2.3, 1.3, 0.3],
[4.4, 3.2, 1.3, 0.2],
[5. , 3.5, 1.6, 0.6],
[5.1, 3.8, 1.9, 0.4],
[4.8, 3. , 1.4, 0.3],
[5.1, 3.8, 1.6, 0.2],
[4.6, 3.2, 1.4, 0.2],
[5.3, 3.7, 1.5, 0.2],
[5. , 3.3, 1.4, 0.2],
[7. , 3.2, 4.7, 1.4],
[6.4, 3.2, 4.5, 1.5],
[6.9, 3.1, 4.9, 1.5],
[5.5, 2.3, 4. , 1.3],
[6.5, 2.8, 4.6, 1.5],
[5.7, 2.8, 4.5, 1.3],
[6.3, 3.3, 4.7, 1.6],
[4.9, 2.4, 3.3, 1. ],
[6.6, 2.9, 4.6, 1.3],
[5.2, 2.7, 3.9, 1.4],
[5. , 2. , 3.5, 1. ],
[5.9, 3. , 4.2, 1.5],
[6. , 2.2, 4. , 1. ],
[6.1, 2.9, 4.7, 1.4],
[5.6, 2.9, 3.6, 1.3],
[6.7, 3.1, 4.4, 1.4],
[5.6, 3. , 4.5, 1.5],
[5.8, 2.7, 4.1, 1. ],
[6.2, 2.2, 4.5, 1.5],
[5.6, 2.5, 3.9, 1.1],
[5.9, 3.2, 4.8, 1.8],
```

```
[6.1, 2.8, 4. , 1.3],
[6.3, 2.5, 4.9, 1.5],
[6.1, 2.8, 4.7, 1.2],
[6.4, 2.9, 4.3, 1.3],
[6.6, 3. , 4.4, 1.4],
[6.8, 2.8, 4.8, 1.4],
[6.7, 3. , 5. , 1.7],
[6. , 2.9, 4.5, 1.5],
[5.7, 2.6, 3.5, 1. ],
[5.5, 2.4, 3.8, 1.1],
[5.5, 2.4, 3.7, 1. ],
[5.8, 2.7, 3.9, 1.2],
[6. , 2.7, 5.1, 1.6],
[5.4, 3. , 4.5, 1.5],
[6. , 3.4, 4.5, 1.6],
[6.7, 3.1, 4.7, 1.5],
[6.3, 2.3, 4.4, 1.3],
[5.6, 3. , 4.1, 1.3],
[5.5, 2.5, 4. , 1.3],
[5.5, 2.6, 4.4, 1.2],
[6.1, 3. , 4.6, 1.4],
[5.8, 2.6, 4. , 1.2],
[5. , 2.3, 3.3, 1. ],
[5.6, 2.7, 4.2, 1.3],
[5.7, 3. , 4.2, 1.2],
[5.7, 2.9, 4.2, 1.3],
[6.2, 2.9, 4.3, 1.3],
[5.1, 2.5, 3. , 1.1],
[5.7, 2.8, 4.1, 1.3],
[6.3, 3.3, 6. , 2.5],
[5.8, 2.7, 5.1, 1.9],
[7.1, 3. , 5.9, 2.1],
[6.3, 2.9, 5.6, 1.8],
[6.5, 3. , 5.8, 2.2],
[7.6, 3. , 6.6, 2.1],
[4.9, 2.5, 4.5, 1.7],
[7.3, 2.9, 6.3, 1.8],
[6.7, 2.5, 5.8, 1.8],
[7.2, 3.6, 6.1, 2.5],
[6.5, 3.2, 5.1, 2. ],
[6.4, 2.7, 5.3, 1.9],
[6.8, 3. , 5.5, 2.1],
[5.7, 2.5, 5. , 2. ],
[5.8, 2.8, 5.1, 2.4],
[6.4, 3.2, 5.3, 2.3],
[6.5, 3. , 5.5, 1.8],
[7.7, 3.8, 6.7, 2.2],
```

```
        [7.7, 2.6, 6.9, 2.3],
        [6. , 2.2, 5. , 1.5],
        [6.9, 3.2, 5.7, 2.3],
        [5.6, 2.8, 4.9, 2. ],
        [7.7, 2.8, 6.7, 2. ],
        [6.3, 2.7, 4.9, 1.8],
        [6.7, 3.3, 5.7, 2.1],
        [7.2, 3.2, 6. , 1.8],
        [6.2, 2.8, 4.8, 1.8],
        [6.1, 3. , 4.9, 1.8],
        [6.4, 2.8, 5.6, 2.1],
        [7.2, 3. , 5.8, 1.6],
        [7.4, 2.8, 6.1, 1.9],
        [7.9, 3.8, 6.4, 2. ],
        [6.4, 2.8, 5.6, 2.2],
        [6.3, 2.8, 5.1, 1.5],
        [6.1, 2.6, 5.6, 1.4],
        [7.7, 3. , 6.1, 2.3],
        [6.3, 3.4, 5.6, 2.4],
        [6.4, 3.1, 5.5, 1.8],
        [6. , 3. , 4.8, 1.8],
        [6.9, 3.1, 5.4, 2.1],
        [6.7, 3.1, 5.6, 2.4],
        [6.9, 3.1, 5.1, 2.3],
        [5.8, 2.7, 5.1, 1.9],
        [6.8, 3.2, 5.9, 2.3],
        [6.7, 3.3, 5.7, 2.5],
        [6.7, 3. , 5.2, 2.3],
        [6.3, 2.5, 5. , 1.9],
        [6.5, 3. , 5.2, 2. ],
        [6.2, 3.4, 5.4, 2.3],
        [5.9, 3. , 5.1, 1.8]]),
 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]),
 'frame': None,
 'target_names': array(['setosa', 'versicolor', 'virginica'], dtype='<U10'),
 'DESCR': '.. _iris_dataset:\n\nIris plants
dataset\n--------------------\n\n**Data Set Characteristics:**\n\n:Number of
Instances: 150 (50 in each of three classes)\n:Number of Attributes: 4 numeric,
predictive attributes and the class\n:Attribute Information:\n    - sepal length
in cm\n    - sepal width in cm\n    - petal length in cm\n    - petal width in
```

```
cm\n    - class:\n            - Iris-Setosa\n            - Iris-Versicolour\n
- Iris-Virginica\n\n:Summary Statistics:\n\n============== ==== ==== =======
===== ====================\n                Min  Max   Mean    SD   Class
Correlation\n============== ==== ==== ======= ===== ====================\nsepal
length:   4.3  7.9   5.84   0.83    0.7826\nsepal width:    2.0  4.4   3.05
0.43   -0.4194\npetal length:   1.0  6.9   3.76   1.76    0.9490  (high!)\npetal
width:    0.1  2.5   1.20   0.76    0.9565  (high!)\n============== ==== ====
======= ===== ====================\n\n:Missing Attribute Values: None\n:Class
Distribution: 33.3% for each of 3 classes.\n:Creator: R.A. Fisher\n:Donor:
Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)\n:Date: July, 1988\n\nThe famous
Iris database, first used by Sir R.A. Fisher. The dataset is taken\nfrom
Fisher\'s paper. Note that it\'s the same as in R, but not as in the
UCI\nMachine Learning Repository, which has two wrong data points.\n\nThis is
perhaps the best known database to be found in the\npattern recognition
literature.  Fisher\'s paper is a classic in the field and\nis referenced
frequently to this day.  (See Duda & Hart, for example.)  The\ndata set contains
3 classes of 50 instances each, where each class refers to a\ntype of iris
plant.  One class is linearly separable from the other 2; the\nlatter are NOT
linearly separable from each other.\n\n.. dropdown:: References\n\n  - Fisher,
R.A. "The use of multiple measurements in taxonomic problems"\n    Annual
Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to\n
Mathematical Statistics" (John Wiley, NY, 1950).\n  - Duda, R.O., & Hart, P.E.
(1973) Pattern Classification and Scene Analysis.\n    (Q327.D83) John Wiley &
Sons.  ISBN 0-471-22361-1.  See page 218.\n  - Dasarathy, B.V. (1980) "Nosing
Around the Neighborhood: A New System\n    Structure and Classification Rule for
Recognition in Partially Exposed\n    Environments".  IEEE Transactions on
Pattern Analysis and Machine\n    Intelligence, Vol. PAMI-2, No. 1, 67-71.\n  -
Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule".  IEEE Transactions\n
on Information Theory, May 1972, 431-433.\n  - See also: 1988 MLC Proceedings,
54-64.  Cheeseman et al"s AUTOCLASS II\n    conceptual clustering system finds 3
classes in the data.\n  - Many, many more …\n',
 'feature_names': ['sepal length (cm)',
  'sepal width (cm)',
  'petal length (cm)',
  'petal width (cm)'],
 'filename': 'iris.csv',
 'data_module': 'sklearn.datasets.data'}
```

[2]:
```
# we only take the first two features for demonstration
x = iris.data[:, :2]
y = iris.target
```

[3]:
```
#print("X array")
#print(x)

#print("\nY array")
#print(y)
```

```
import pandas as pd

df_x = pd.DataFrame(x, columns=iris.feature_names[:2])
display(df_x)
```

```
     sepal length (cm)  sepal width (cm)
0                  5.1               3.5
1                  4.9               3.0
2                  4.7               3.2
3                  4.6               3.1
4                  5.0               3.6
..                 …                 …
145                6.7               3.0
146                6.3               2.5
147                6.5               3.0
148                6.2               3.4
149                5.9               3.0

[150 rows x 2 columns]
```

The target class is the species of the iris flower. It can be one of three values: setosa, versicolor, or virginica.

[4]:
```
df_y = pd.DataFrame(y, columns=["target"])
df_y['Flower name'] = df_y['target'].map({0: 'setosa', 1: 'versicolor', 2:␣
↪'virginica'})
display(df_y)
```

```
     target Flower name
0         0      setosa
1         0      setosa
2         0      setosa
3         0      setosa
4         0      setosa
..      …          …
145       2   virginica
146       2   virginica
147       2   virginica
148       2   virginica
149       2   virginica

[150 rows x 2 columns]
```

[5]:
```
#number_of_neighbours_to_test = (3,15) # Just 3 and 15
#number_of_neighbours_to_test = range(1, 16) # All numbers from 3 to 15
number_of_neighbours_to_test = range(1, 20)
fitted_algorithms = {}
```

```
for k in number_of_neighbours_to_test:
    clf = neighbors.KNeighborsClassifier(n_neighbors=k)
    clf.fit(x, y)
    fitted_algorithms[k] = clf
```

Plotting the fitted algorithms

[6]:
```
import matplotlib.pyplot as plt
from sklearn.inspection import DecisionBoundaryDisplay
from matplotlib.colors import ListedColormap

region_cmap = ListedColormap(["orange", "cyan", "cornflowerblue"])
point_cmap = ListedColormap(["orange", "cyan", "navy"])
use_light_colours_bool = False

fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharex=True, sharey=True)

for ax, (k, clf) in zip(axes, ((k, fitted_algorithms[k]) for k in (15, 3))):
    if use_light_colours_bool:
        DecisionBoundaryDisplay.from_estimator(
                clf,
                x,
                response_method="predict",
                cmap="Pastel1",
                alpha=0.6,
                ax=ax,
            )

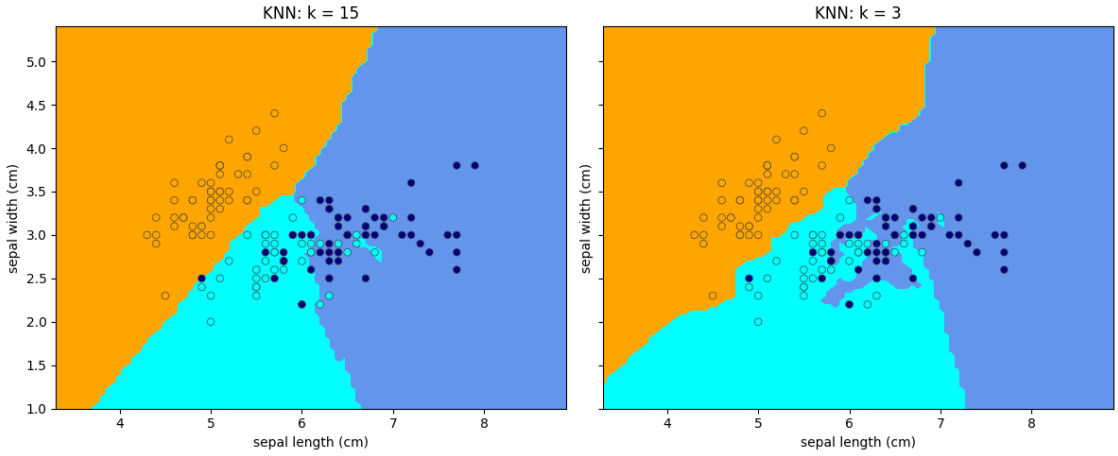
        ax.scatter(
            x[:, 0], x[:, 1],
            c=y,
            cmap="Set1",
            edgecolors="black",
            s=35,
        )
    else:
        DecisionBoundaryDisplay.from_estimator(
            clf,
            x,
            response_method="predict",
            cmap=region_cmap,
            alpha=1,
            ax=ax,
        )

        ax.scatter(
```

```
            x[:, 0], x[:, 1],
            c=y,
            cmap=point_cmap,
            vmin=0,
            vmax=2,
            edgecolors="#444444",
            linewidths=0.5,
            s=30,
        )

    ax.set_title(f"KNN: k = {k}")
    ax.set_xlabel(iris.feature_names[0])
    ax.set_ylabel(iris.feature_names[1])

plt.tight_layout()
plt.show()
```



[7]:
```
import math
import matplotlib.pyplot as plt
from sklearn.inspection import DecisionBoundaryDisplay

n_models = len(fitted_algorithms)
n_rows = math.ceil(n_models / 2)

fig, axes = plt.subplots(
    n_rows, 2,
    figsize=(12, 5 * n_rows),
    sharex=True,
    sharey=True,
    squeeze=False,
)
```

```
axes = axes.ravel()

for ax, (k, clf) in zip(axes, fitted_algorithms.items()):
    if use_light_colours_bool:
        DecisionBoundaryDisplay.from_estimator(
            clf,
            x,
            response_method="predict",
            cmap="Pastel1",
            alpha=0.6,
            ax=ax,
        )

        ax.scatter(
            x[:, 0], x[:, 1],
            c=y,
            cmap="Set1",
            edgecolors="black",
            s=35,
        )
    else:
        DecisionBoundaryDisplay.from_estimator(
            clf,
            x,
            response_method="predict",
            cmap=region_cmap,
            alpha=1,
            ax=ax,
        )

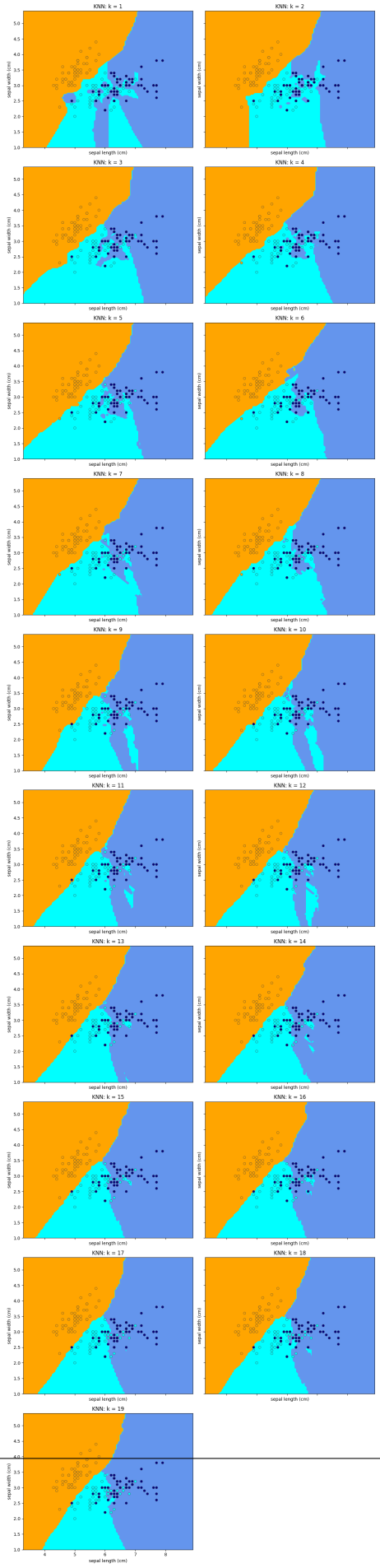
        ax.scatter(
            x[:, 0], x[:, 1],
            c=y,
            cmap=point_cmap,
            vmin=0,
            vmax=2,
            edgecolors="#444444",
            linewidths=0.5,
            s=30,
        )

    ax.set_title(f"KNN: k = {k}")
    ax.set_xlabel(iris.feature_names[0])
    ax.set_ylabel(iris.feature_names[1])

# Hide the extra subplot when the model count is odd.
for ax in axes[n_models:]:
```

```
    ax.set_visible(False)

plt.tight_layout()
plt.show()
```


KNN: k = 1
KNN: k = 2
KNN: k = 3
KNN: k = 4
KNN: k = 5
KNN: k = 6
KNN: k = 7
KNN: k = 8
KNN: k = 9
KNN: k = 10
KNN: k = 11
KNN: k = 12
KNN: k = 13
KNN: k = 14
KNN: k = 15
KNN: k = 16
KNN: k = 17
KNN: k = 18
KNN: k = 19
sepal length (cm)
sepal width (cm)

1.0.1 Why are there 3 colours regardless of k?

The **3 colours represent the 3 Iris species**: setosa, versicolor, and virginica.

In K-Nearest Neighbours (KNN), **k is the number of nearby training samples used to make a prediction**, not the number of groups.

For any point on the plot:

1. KNN finds its **k nearest training samples**.
2. Those samples vote for their species.
3. The **background colour** shows the predicted species, while each **dot's colour** shows its actual species.

Changing k changes the **shape of the decision boundaries**, but there are still only 3 possible species, so there are still 3 colours.

- **Smaller k:** usually produces more irregular boundaries and is more sensitive to noise.
- **Larger k:** generally produces smoother boundaries but may miss local patterns.

KNN versus K-means: In KNN, k is the number of neighbours consulted. In K-means, k is the number of clusters to create.

1.1 Which algorighm performs best.

Compare their predictions on **held-out data**—samples they didn't train on. The plots alone won't tell you which performs best.

For this Iris example, use the same training and validation split for every value of `k`:

[8]:
```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier

x_train, x_val, y_train, y_val = train_test_split(
    x, y,
    test_size=0.25,
    random_state=42,
    stratify=y,
)

scores = {}

for k in number_of_neighbours_to_test:
    clf = KNeighborsClassifier(n_neighbors=k)
    clf.fit(x_train, y_train)

    predictions = clf.predict(x_val)
    scores[k] = accuracy_score(y_val, predictions)
```

```python
for k, score in sorted(scores.items(), key=lambda item: item[1], reverse=True):
    print(f"k = {k:2}: validation accuracy = {score:.1%}")
```

```
k = 13: validation accuracy = 84.2%
k =  9: validation accuracy = 81.6%
k = 10: validation accuracy = 81.6%
k = 11: validation accuracy = 81.6%
k = 17: validation accuracy = 78.9%
k = 19: validation accuracy = 78.9%
k = 14: validation accuracy = 76.3%
k = 15: validation accuracy = 76.3%
k = 16: validation accuracy = 76.3%
k = 18: validation accuracy = 73.7%
k =  6: validation accuracy = 71.1%
k =  8: validation accuracy = 71.1%
k = 12: validation accuracy = 71.1%
k =  5: validation accuracy = 68.4%
k =  7: validation accuracy = 68.4%
k =  1: validation accuracy = 65.8%
k =  2: validation accuracy = 65.8%
k =  4: validation accuracy = 65.8%
k =  3: validation accuracy = 60.5%
```